Messstationen-Pipeline (PostGIS + Python)

Probearbeitstag ZEBRIS GmbH, München, 28.10.2026 · Gokul Gopan

5.010	4.900	110	4	19
Zeilen gelesen	Stationen übernommen	mit Grund verworfen	Gebiete	Tests bestanden

1. Aufgabe und Lösung im Überblick

Teil	Anforderung	Umsetzung
A	Nur brauchbare Datensätze in `stationen`, verworfene mit Grund in `stationen_abgelehnt`, Geometriespalte mit Index, wiederholbar ohne Duplikate, `gebiete` laden	`pipeline.py`, `tabellen_vorbereiten.sql`
B	Stationen je Gebiet; nächste Station mit Abstand in Metern und Anzahl im Umkreis von 500 m; mittleres PM10 je Gebiet und Typ	`auswertung.sql` (3 Views)
C	`python pipeline.py stationen.csv gebiete.geojson` mit Bericht, mindestens 3 Tests	`pipeline.py`, `test_pipeline.py` (19 Tests)
D (optional)	PyQGIS: `stationen` laden, nach `typ` kategorisieren, GeoPackage mit CRS exportieren	`qgis_export_v1.py`, `qgis_export_v2.py`

2. Arbeitsumgebung

Komponente	Version	Verwendung
PostgreSQL (Postgres.app, lokal)	16.15	Datenbank `zebris_task` auf localhost:5432
PostGIS	3.4.6	Geometrien, GiST-Indizes, Auswertung
GDAL / OGR	3.7.3	`ogrinfo` zur Datensichtung, `ogr2ogr` zum Laden
Python (venv)	3.12.7	Pipeline und Tests in einer eigenen virtuellen Umgebung
GeoPandas · Shapely · pyproj · pyogrio	1.2.0 · 2.2.0 · 3.8.0 · 0.13.0	Lageprüfung, Transformation, GeoPackage schreiben
QGIS	3.44.13	Sichtprüfung, PyQGIS (Teil D)
DBeaver		Tabellen, Indizes und Ergebnisse in der Datenbank prüfen

Docker ist auf dem Rechner vorhanden. Für diese Aufgabe habe ich bewusst die lokale PostgreSQL-Installation verwendet, weil sie bereits eingerichtet war und keine zusätzliche Schicht für Ports, Volumes und Netz nötig war.

Zugangsdaten: In keinem Skript steht ein Passwort. Die lokale Datenbank akzeptiert Verbindungen von localhost ohne Passwort (Postgres.app, Authentifizierung `trust`). Die Pipeline liest die Verbindung aus den Umgebungsvariablen `PGHOST`, `PGPORT`, `PGDATABASE`, `PGUSER`, `PGPASSWORD` oder aus `~/.pgpass`; das PyQGIS-Skript (Version 2) nutzt die in QGIS gespeicherte Verbindung.

3. Arbeitsablauf

1. **Datensichtung mit GDAL und QGIS** eigene Arbeit
   Beide Dateien mit `ogrinfo -so` und SQL direkt auf der CSV (`-dialect SQLite`) geprüft; CSV als Textebene in QGIS über die Gebiete gelegt. Befunde: BOM, Semikolon, CRLF, Dezimalkommas, vertauschte und fehlende Koordinaten, Platzhalter bei PM10, zwei Datumsformate, doppelte IDs. Festgehalten in `datensichtung.md`.

2. **Fachliche Entscheidungen** eigene Arbeit
   Speicherung in EPSG:25832 (Meter, passend zu den Gebieten); ungültiges Datum und PM10-Platzhalter werden NULL, die Station bleibt; Lageprüfung gegen die gelieferten Gebietspolygone statt fester Koordinatengrenzen; Verarbeitung nur in Python (GeoPandas), Laden nur mit `ogr2ogr`.

3. **Datenanalyse und Pipeline-Code** KI-unterstützt
   Bereinigungsregeln, Duplikaterkennung, Bericht und Tests mit KI-Unterstützung ausgearbeitet: Claude Code, Modell Claude Opus 5.5 (Einstellung „medium“). Jede Regel habe ich an den Zählungen aus Schritt 1 nachvollzogen.

4. **Code-Review und Prüfung** eigene Arbeit KI-unterstützt
   Zweite KI für das abschließende Review und den Vergleich der Fassungen: ChatGPT, Modell Astra (Einstellung „high“); jeden Befund vor einer Änderung geprüft. Alle Skripte gelesen, Tests ausgeführt, Ergebnis mit Prüfsummen über zwei Läufe verglichen.

5. **Laden in PostGIS** eigene Arbeit
   `python pipeline.py stationen.csv gebiete.geojson`, danach `psql -f auswertung.sql`.

6. **Kontrolle in DBeaver** eigene Arbeit
   Spalten, Geometrietyp, Indizes und die drei Auswertungen direkt in der Datenbank abgefragt (Abschnitt 8).

7. **Visualisierung in QGIS (Teil D)** eigene Arbeit KI-unterstützt
   Version 1 des PyQGIS-Skripts selbst im Stil meines PyQGIS-Kurses geschrieben und ausgeführt; Version 2 mit KI-Unterstützung robuster gemacht (Abschnitt 7).

4. Teil A: Laden und Bereinigen

Ablauf in `pipeline.py`: CSV einlesen (`utf-8-sig`, `;`, Kopf und Feldanzahl geprüft) → jedes Feld bereinigen → Lage mit GeoPandas gegen die Gebiete prüfen → Duplikate auflösen → Zeilenbilanz prüfen → GeoPackage und CSV in ein temporäres Verzeichnis schreiben → Tabellen vorbereiten → mit `ogr2ogr -append` laden.

Regeln

Befund	Regel
Koordinate fehlt oder ist 0	verwerfen
Dezimalkomma; fehlender Dezimalpunkt (z. B. 541073)	reparieren (54.1073), danach räumlich prüfen
lat/lon vertauscht	beide Achsenreihenfolgen gegen die Gebiete prüfen: nur vertauscht passend → tauschen; keine passend → verwerfen; beide passend → als mehrdeutig verwerfen
unmögliches Datum (30.02.)	NULL, Station bleibt
PM10 –999, n/a, leer, nicht endlich	NULL, Station bleibt; zählt nicht in den Mittelwert
Leerzeichen in `station_id`	entfernen, danach Duplikate prüfen
gleiche ID, identische Werte	eine Zeile behalten
gleiche ID, unterschiedliche Werte	beide verwerfen (keine Version ist belegbar richtig)
gleiche Station unter anderer ID („Messcontainer“)	gerundete Lage, Betreiber, Typ, Datum und PM10 gleich → kleinste ID behalten

Ergebnis

Ablehnungsgrund	Anzahl	Korrektur bei übernommenen Stationen	Anzahl
Koordinate fehlt	25	Dezimalkomma ersetzt	60
doppelte Zeile	20	ungültiger PM10-Wert → NULL	49
gleiche ID, unterschiedliche Werte	20	lat/lon getauscht	40
Dublette unter anderer ID	20	Dezimalpunkt ergänzt	5
Koordinate ist 0	15	Datum ungültig → NULL	5
doppelte Zeile nach Bereinigung	10	Leerzeichen in ID entfernt	5
Summe	**110**	**4.900 + 110 = 5.010 (Zeilenbilanz im Code geprüft)**	

Datenbank

Tabelle	Struktur
`gebiete`	`gebiet_id` PK, `geom geometry(Polygon, 25832)`, GiST-Index
`stationen`	`station_id` PK, `pm10_ugm3 numeric(6,1)`, `inbetriebnahme date`, `geom geometry(Point, 25832) NOT NULL`, GiST-Index, Spalte `korrektur`
`stationen_abgelehnt`	Zeilennummer, ID, Grund, Hinweis, Rohdaten als JSON

Wiederholbar: `tabellen_vorbereiten.sql` legt die Tabellen mit `CREATE TABLE IF NOT EXISTS` an und leert sie vor dem Laden. Zwei aufeinanderfolgende Läufe ergaben identische Prüfsummen über alle Stationen und Ablehnungen.

5. Teil B: Auswertung

Drei Views in `auswertung.sql`. Punkt im Gebiet mit `ST_Covers` (Punkte auf der Grenze zählen, wie in der Python-Prüfung). Nächste Station per KNN-Operator `<->` in einem `LEFT JOIN LATERAL`, 500-m-Umkreis mit

`ST_DWithin`, das den GiST-Index nutzt. Abstände sind echte Meter, weil die Daten in EPSG:25832 liegen.

Gebiet	Stationen	PM10 Hintergrund	PM10 Industrie	PM10 Verkehr
G1 Nordwest	1.809	18,1	18,3	26,0
G2 Nordost	300	18,9	18,8	25,6
G3 Südwest	1.675	17,6	18,4	25,8
G4 Südost	1.116	17,7	18,0	26,1

PM10 in µg/m³. 128 Stationen haben keinen PM10-Wert; `avg()` ignoriert NULL, die View zeigt daher zusätzlich „Stationen mit Wert“ und „Stationen gesamt“. Kleinster Abstand zwischen zwei Stationen nach der Bereinigung: 7,9 m (vorher 0 m durch die Dubletten unter anderer ID).

6. Teil C: Programm und Tests

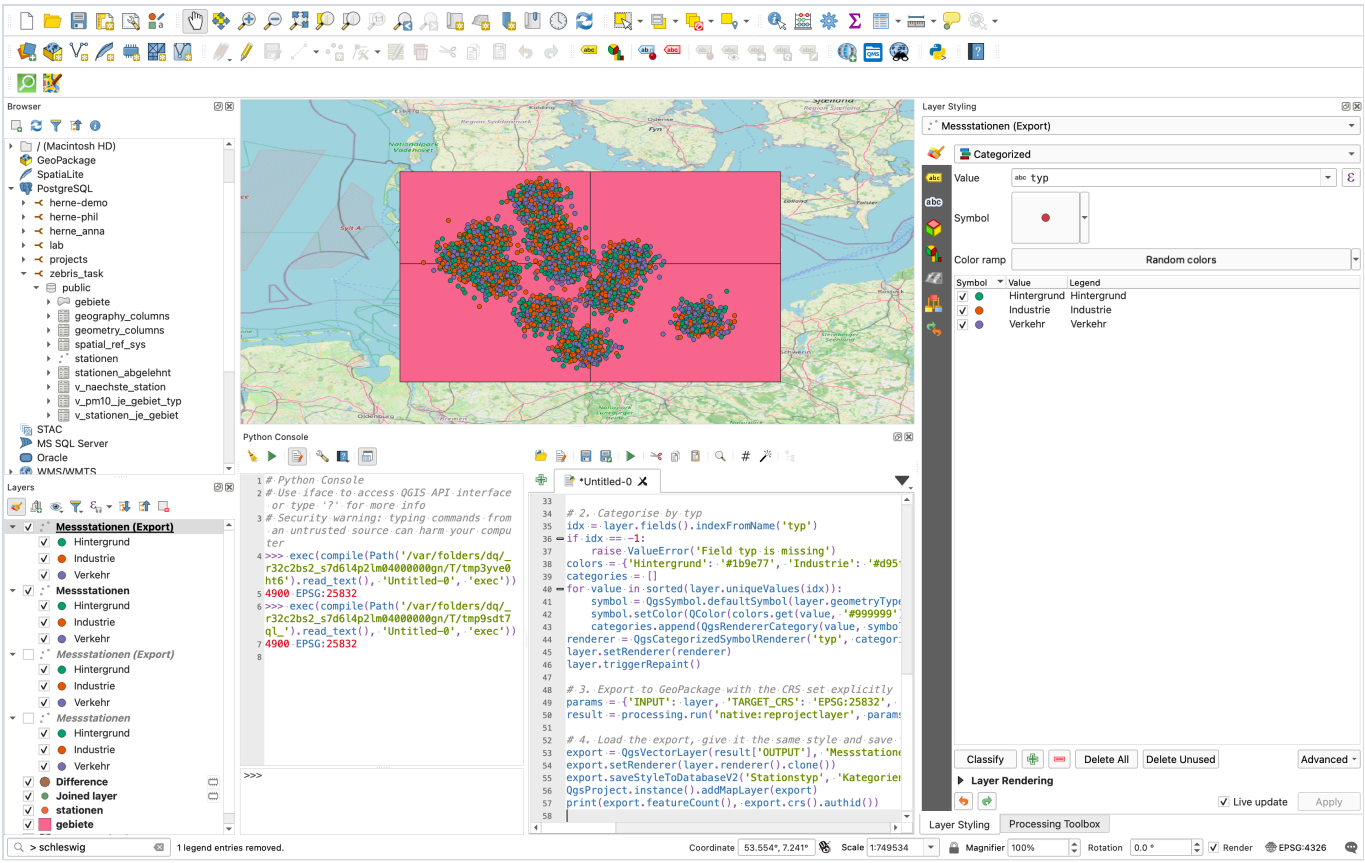
```
python pipeline.py stationen.csv gebiete.geojson
python -m unittest -v test_pipeline        # 19 Tests, OK
```

Der Bericht zeigt gelesene, übernommene und verworfene Zeilen mit Gründen sowie die Korrekturen. Die Tests decken Dezimalkomma und fehlenden Dezimalpunkt, fehlende und Null-Koordinaten, vertauschte Achsen mit echter Geometrieprüfung, Datumsformate, PM10-Platzhalter, alle Duplikatfälle, `nan` / `inf`, fehlende CSV-Spalten und leere Ausgaben ab. Sie laufen ohne Datenbank.

7. Teil D: PyQGIS

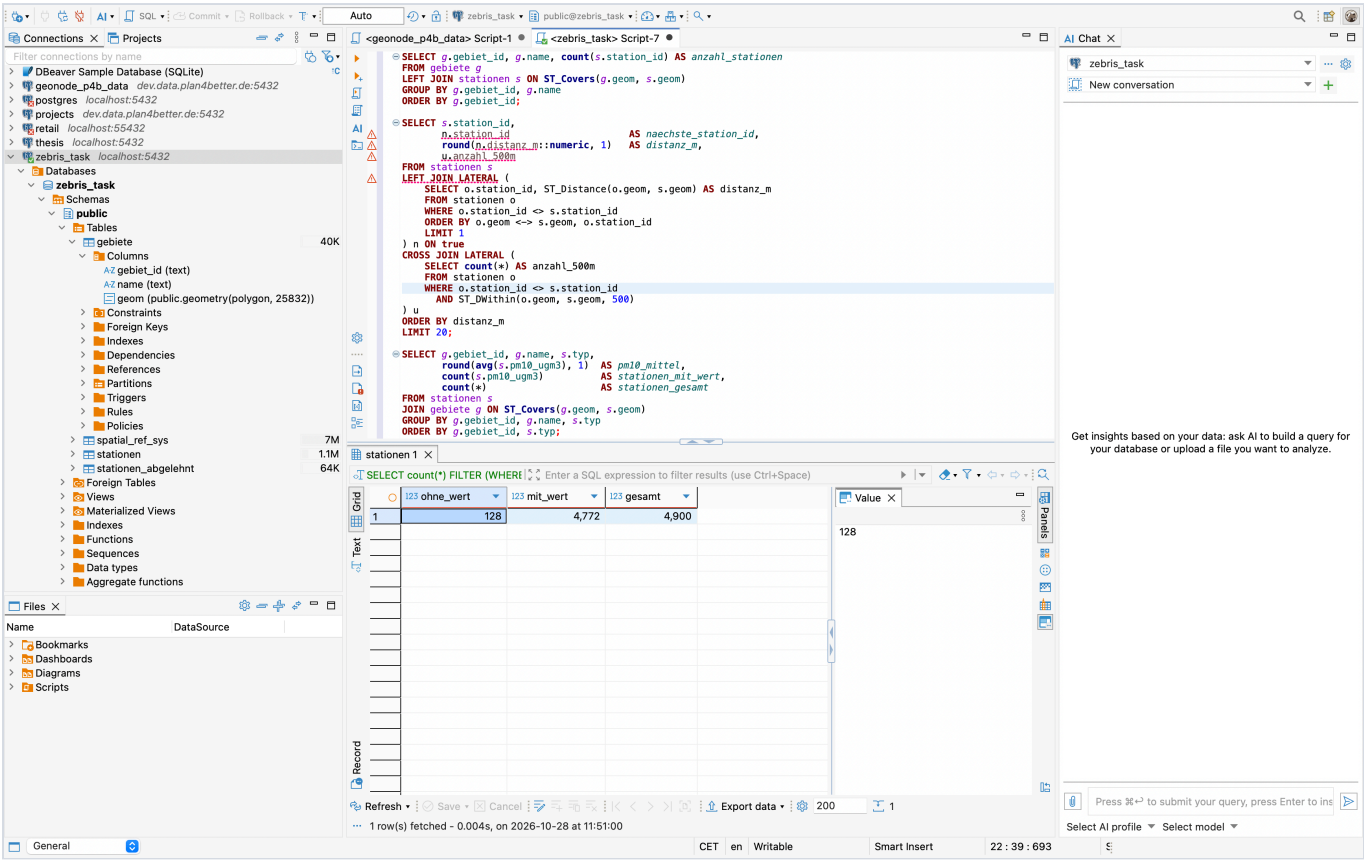
Version 1 (`qgis_export_v1.py`, eigene Fassung im Stil des Spatial-Thoughts-PyQGIS-Kurses): PostGIS-Layer laden, nach `typ` kategorisieren, mit `native:reprojectlayer` als GeoPackage in EPSG:25832 schreiben, Stil mit `saveStyleToDatabaseV2` in der Tabelle `layer_styles` des GeoPackage speichern. Ergebnis: 4.900 Stationen, EPSG:25832.

Version 2 (`qgis_export_v2.py`, mit KI-Unterstützung aus Version 1): gespeicherte QGIS-Verbindung statt Host und Benutzer im Code, kein Überschreiben vorhandener Ausgaben, Prüfung des Feldes `typ`, genauere Fehlertypen. Unbekannte Typen werden grau dargestellt.



QGIS 3.44: Gebiete (rot) und die exportierten Stationen nach Typ. Rechts der in das GeoPackage übernommene Kategorienstil, unten die Konsolenausgabe „4900 EPSG:25832“.

8. Prüfung in DBeaver



DBeaver auf `zebris_task` : `gebiete.geom` als `geometry(Polygon, 25832)` , die drei Auswertungen als Abfragen, Kontrolle der PM10-Werte (128 ohne Wert, 4.772 mit Wert, 4.900 gesamt).

9. Einsatz von KI

KI war für die Aufgabe erlaubt. Ich habe sie gezielt für einzelne Schritte eingesetzt, nicht für den gesamten Ablauf. Alle Werkzeuge liefen lokal im Editor VS Code.

Schritt	Werkzeug	Was die KI gemacht hat · was ich gemacht habe
Datensichtung	GDAL, QGIS	Befunde mit `ogrinfo` und in QGIS selbst ermittelt und dokumentiert.
Datenanalyse und Pipeline	Claude Code, Claude Opus 5.5 („medium“)	KI: Ausarbeitung des Python-Codes, der Tests und der SQL-Views. Ich: Vorgaben (Python/GeoPandas, `ogr2ogr` , EPSG:25832, NULL-Regeln, Lageprüfung gegen Gebiete), jede Regel an den Daten nachvollzogen.
Code-Review	ChatGPT, Modell Astra („high“)	KI: abschließendes Review und Vergleich der Fassungen mit Verbesserungsvorschlägen (u. a. `inf` / `nan` , Spaltenprüfung). Ich: Befunde einzeln geprüft, nur bestätigte übernommen.
Ausführen und Prüfen	Terminal, DBeaver, QGIS	Pipeline ausgeführt, Ergebnisse in DBeaver abgefragt, in QGIS visuell geprüft.
PyQGIS	QGIS-Konsole; Claude Code	Version 1 selbst geschrieben und ausgeführt; Version 2 mit KI-Unterstützung erweitert.
Dokumentation	Claude Code	Entwurf mit KI-Unterstützung, Inhalte und Zahlen von mir geprüft.

10. Grenzen

- **Keine gemeinsame Transaktion beim Laden.** Die Tabellen werden geleert, danach laden drei `ogr2ogr`-Aufrufe. Bricht ein Aufruf ab, bleibt ein unvollständiger Stand, bis der Lauf wiederholt wird. Alle Dateien werden vorher geschrieben, Fehler beim Lesen und Bereinigen verändern die Datenbank also nicht. Nächster Schritt: in Staging-Tabellen laden und in einer Transaktion übernehmen.
- **Heuristiken.** Die Reparatur des fehlenden Dezimalpunkts und der Dublettenschlüssel (gerundete Gradwerte) passen zu diesen Daten und wären für andere Lieferungen fachlich zu bestätigen.
- **Untersuchungsraum.** Stationen außerhalb der vier Gebiete werden verworfen; das ist eine Annahme für diese Aufgabe.

11. Reproduzieren

Voraussetzungen: Python 3.10 oder neuer, PostgreSQL mit PostGIS, GDAL mit `ogr2ogr` und `psql` im PATH, für Teil D QGIS 3.28 oder neuer.

```
git clone <repository-url> && cd <ordner>
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export PGHOST=localhost PGDATABASE=zebris_task
createdb "$PGDATABASE"
psql -X -v ON_ERROR_STOP=1 -c 'CREATE EXTENSION IF NOT EXISTS postgis'
python pipeline.py stationen.csv gebiete.geojson
psql -X -v ON_ERROR_STOP=1 -f auswertung.sql
python -m unittest -v test_pipeline
```

Teil D: in QGIS unter PostgreSQL eine Verbindung `zebris_task` anlegen, dann `qgis_export_v2.py` im Editor der Python-Konsole öffnen, `OUTPUT` anpassen und starten.